

Node.js & Express - Lecture Notes

Node.js Event Loop & Architecture

Event Loop Behavior

- When the outer loop runs once, it waits at the **poll stage**
- Network controls and file system reads all happen at the **poll stage**
- The thread goes outside to get resources from the system
- When resources are available, Node.js starts streaming the data
- This is why Node.js is called **semi-asynchronous** (or "semi-eleptical" as noted)

Poll Phase

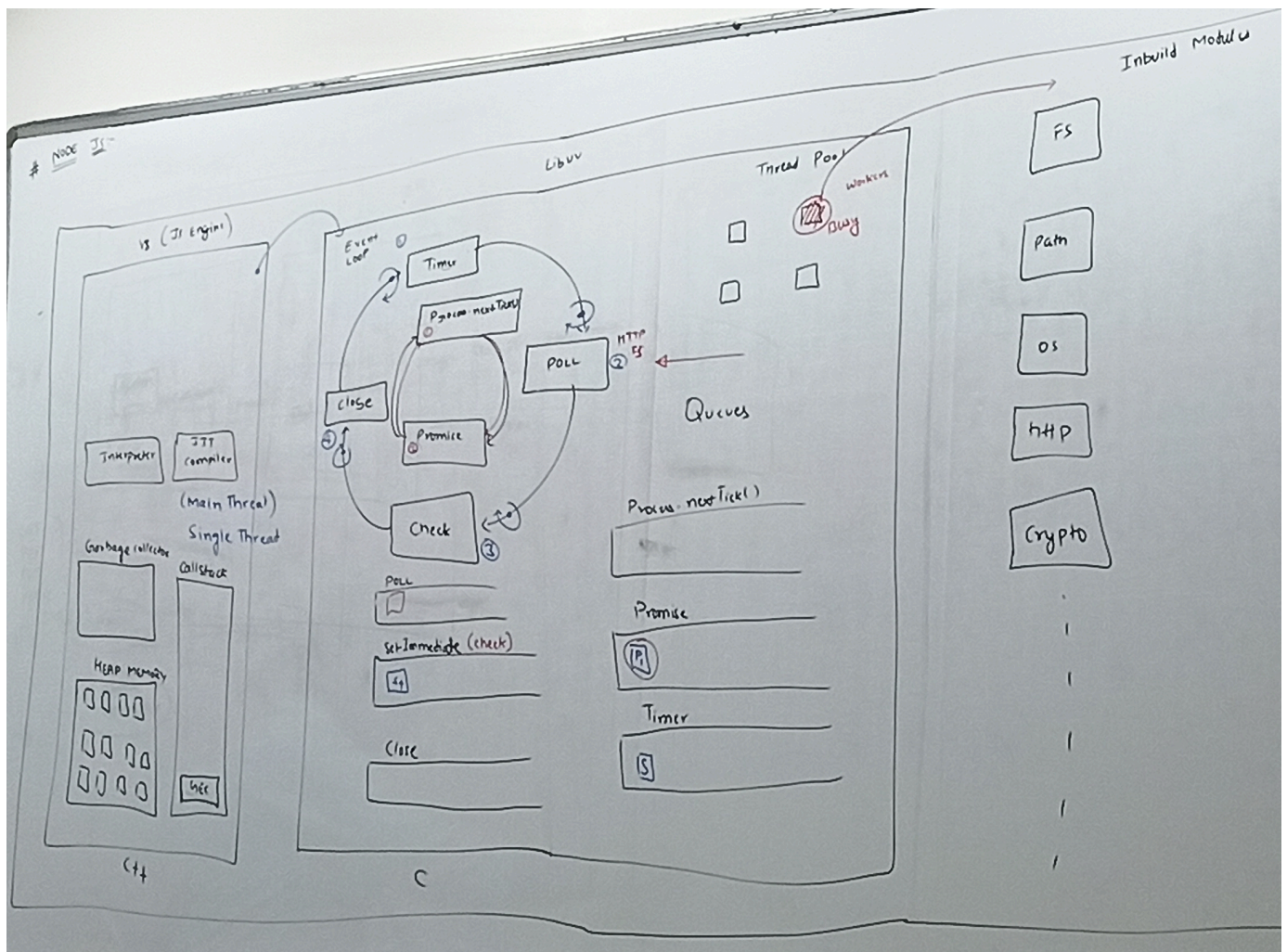
- The poll stage checks for timer phases
- Before moving to another phase, inner phases are checked in **libuv**
- This ensures proper event handling and callback execution

Thread Pool Configuration

- Default thread pool size can be increased using environment variables
- In your `.env` file, add:

```
UV_THREADPOOL_SIZE=10
```

- This increases the number of threads available for async operations



Types of Modules in Node.js

1. Built-in Modules

- Core modules that come with Node.js
- Examples: `fs`, `http`, `path`, `os`, etc.

2. User-Defined Modules

- Custom modules created by developers
- Exported using `module.exports` or `export`

3. Third-Party Modules

- External packages installed via npm
- Examples: `express`, `mongoose`, `dotenv`, etc.

Express Framework

What is Express?

- Express is a **web application framework** for Node.js
- Minimal and flexible framework for building web applications
- Simplifies routing, middleware, and server setup

Key Features

- Easy database connection and integration
 - Simplified HTTP request handling
 - Robust routing system
 - Middleware support
-

Middleware in Express

1. `app.use()`

- Used to handle **middleware functions**
- Handles **all HTTP methods**: GET, POST, PUT, PATCH, DELETE
- Example:

```
app.use('/api', middlewareFunction);
```

- URL matching:
 - `http://localhost:3000/api/user` ✅ (matches)
 - `http://localhost:3000/api/products` ✅ (matches)
 - Any route starting with `/api` will match

2. `app.all()`

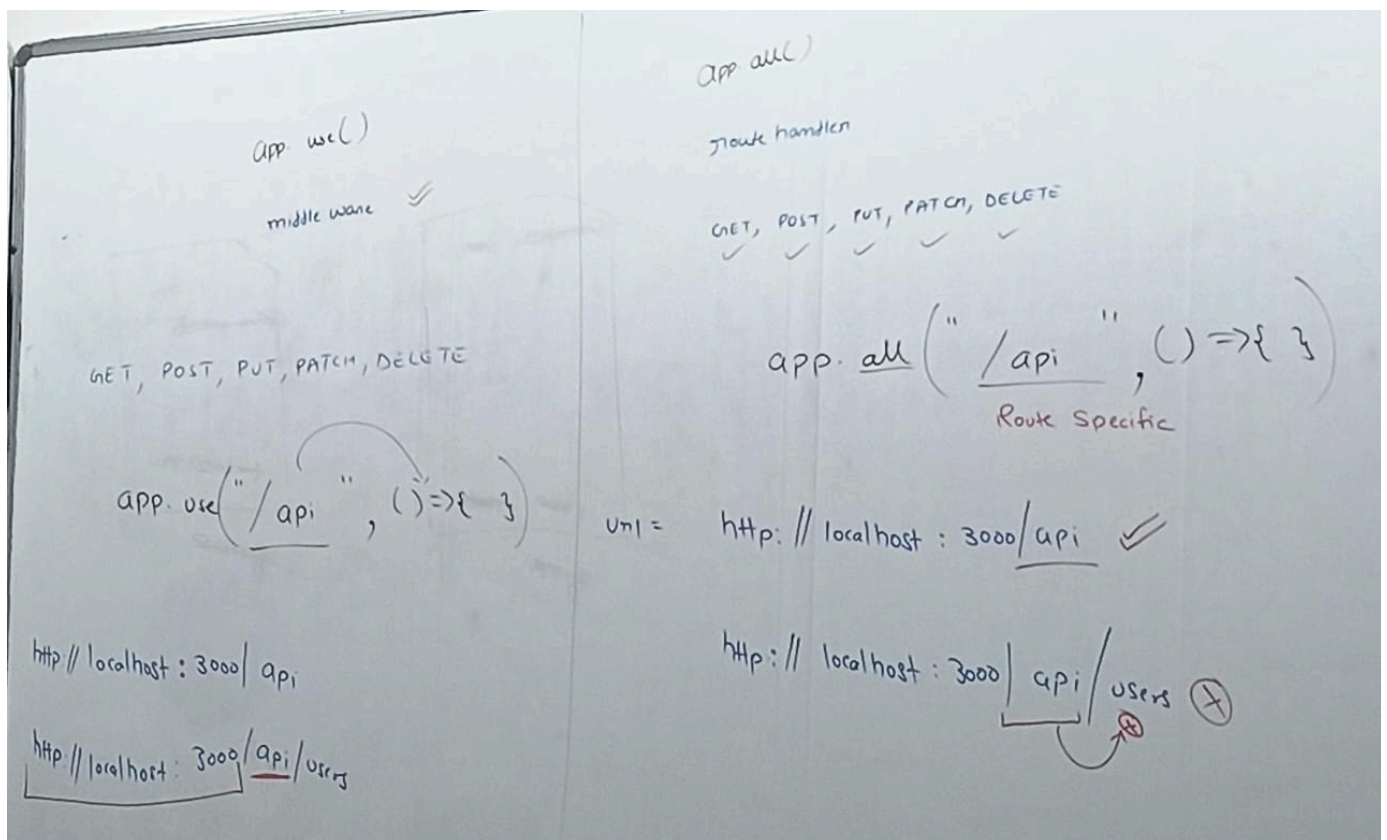
- Used to handle a **specific route**
- Handles **all HTTP methods**: GET, POST, PUT, PATCH, DELETE
- **Route-specific** - exact match only
- Example:

```
app.all('/api', (req, res) => {  
  // handler  
});
```

- URL matching:
 - `http://localhost:3000/api` ✓ (matches)
 - `http://localhost:3000/api/user` ✗ (does not match)
 - Only exact `/api` route is valid

Difference Between `app.use()` and `app.all()`

Feature	<code>app.use()</code>	<code>app.all()</code>
Purpose	Middleware mounting	Route handling
Path matching	Prefix matching	Exact matching
Use case	Apply middleware to multiple routes	Handle all methods on specific route



Types of Middleware

1. User-Defined Middleware

a) Global Middleware

- Applied to all routes in the application
- Example:

```
app.use((req, res, next) => {
  console.log('Request received');
  next();
});
```

b) Route-Specific Middleware

- Applied to specific routes only
- Example:

```
const authMiddleware = (req, res, next) => {
  // authentication logic
  next();
};

app.get('/api/user', authMiddleware, (req, res) => {
  res.send('User data');
});
```

c) Error Middleware

- Handles errors in the application
- Must have 4 parameters: `(err, req, res, next)`
- Example:

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something went wrong!');
});
```

2. Built-in Middleware

`express.json()`

- Parses incoming requests with JSON payloads
- Example:

```
app.use(express.json());
```

`express.urlencoded()`

- Parses incoming requests with URL-encoded payloads (form data)
- Example:

```
app.use(express.urlencoded({ extended: true }));
```

`express.static()`

- Serves static files (images, CSS, JavaScript)
- Example:

```
app.use(express.static('public'));
```

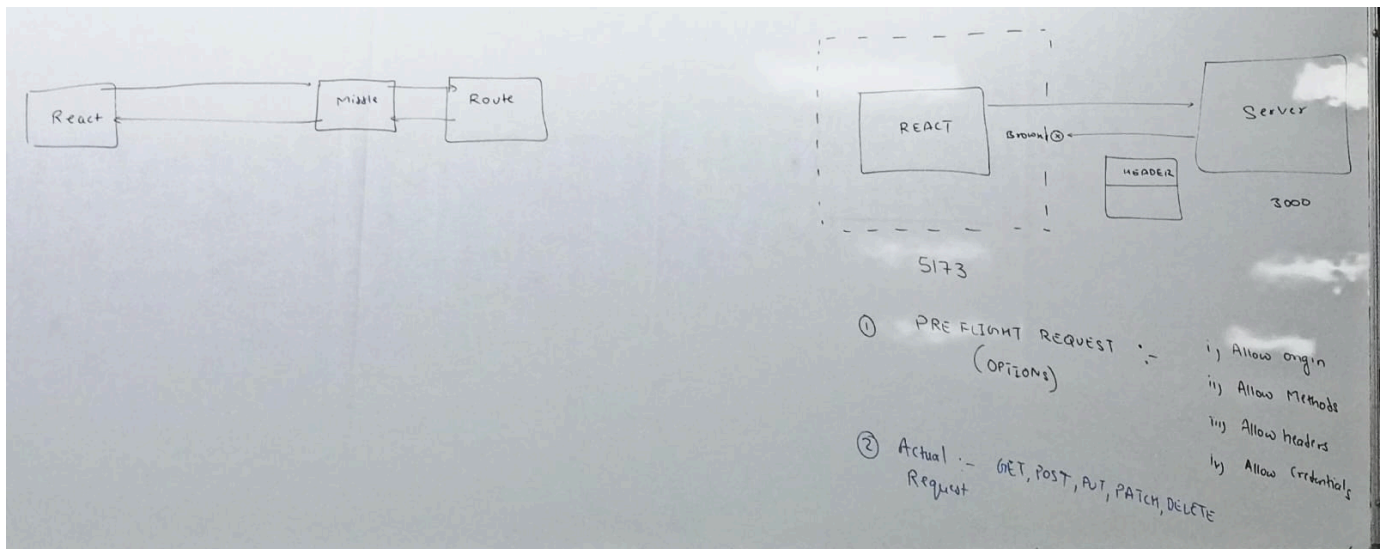
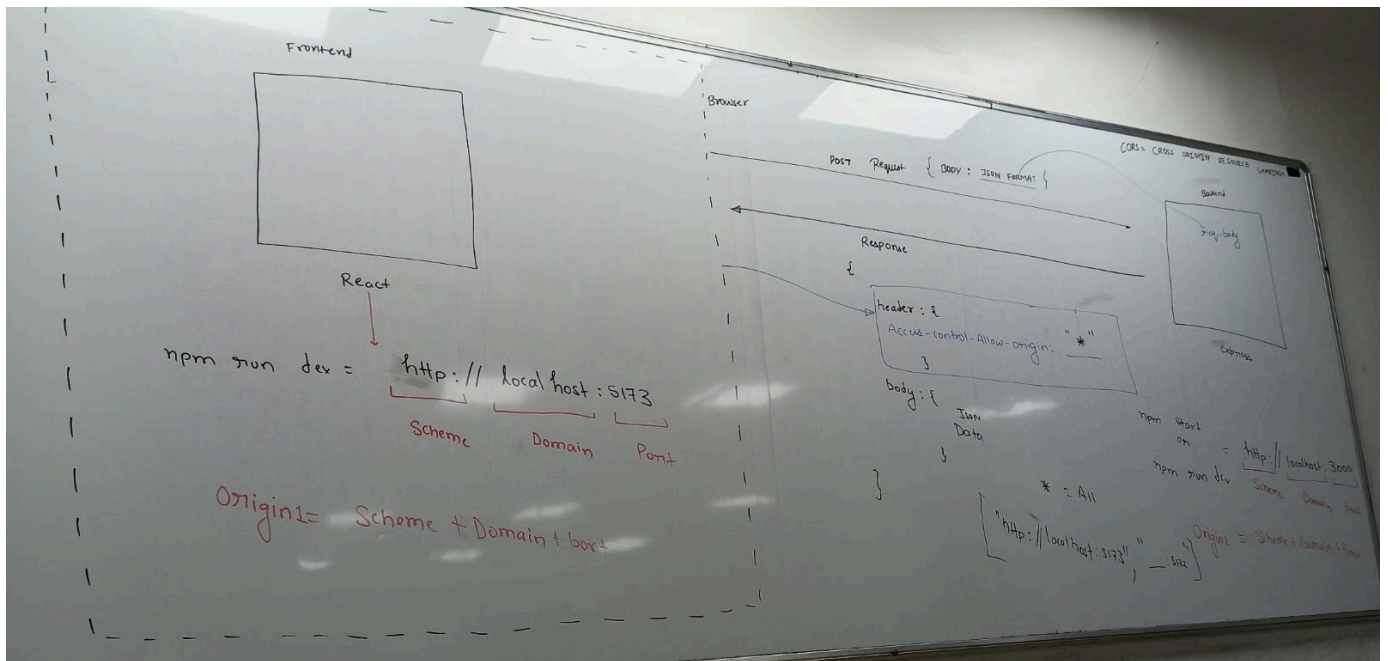
3. Third-Party Middleware

Popular third-party middleware packages:

- **cors**: Enable Cross-Origin Resource Sharing
- **csrf**: CSRF protection (note: typo in original - "csruf")
- **multer**: Handle multipart/form-data (file uploads)
- **passport**: Authentication middleware
- **cookie-parser**: Parse cookies
- **express-session**: Session management
- **jwt** (jsonwebtoken): JSON Web Token authentication

Example Usage

```
const cors = require('cors');  
const cookieParser = require('cookie-parser');  
  
app.use(cors());  
app.use(cookieParser());
```

```
Request → Global Middleware → Route-Specific Middleware → Route Handler → Response
      ↓ (if error)
      Error Middleware
```

1. Always call `next()` in middleware unless sending a response
2. Place error middleware at the end of middleware stack
3. Use built-in middleware before custom middleware
4. Keep middleware functions focused and single-purpose
5. Use environment variables for configuration (like thread pool size)

- Middleware order matters - they execute in the order they're defined
- Global middleware affects all routes that come after it
- Error middleware should always be defined last
- Use `next(error)` to pass errors to error handling middleware